

软工二期末名词解释

目录

1	软件工程与计算 II 期末名词解释	2
2	一、总图：先记住软件工程活动链	2
3	二、为什么需要软件工程	3
4	三、项目管理与过程控制	4
5	四、需求工程：把现实问题变成规格	6
6	五、体系结构与人机交互：把规格组织成系统	10
7	六、详细设计与模块化：把架构落到类和对象	13
8	七、设计模式：用稳定抽象隔离变化	17
9	八、软件构造、代码质量与测试	18
10	九、AI 协作、工具链与 Web 实现	22
11	十、卷面速记：相似概念判定句	23

1 软件工程与计算 II 期末名词解释

1.1 使用说明

本文件按 PPT/ 课件和软工二课程整理/课程框架与重点名词.md 重新梳理。背诵顺序不按原来的零散名词走，而按软件工程活动链路走：

为什么需要工程
-> 怎样管理项目
-> 怎样把现实问题写成需求
-> 怎样把需求组织成架构
-> 怎样把架构细化成类和模块
-> 怎样写出可维护代码并测试
-> AI 和工具链如何贯穿全过程

记忆方法：

- 先背每章的“记忆链”，知道概念之间的上下位关系。
- 再背名词定义，答题时按“是什么 + 解决什么问题 + 常见关键词”写。
- 最后背卷面判定句，用来区分相似概念。

整理依据：

PPT/00-00-课程Overview.pdf
PPT/01-01-软件工程基础.pdf
PPT/01-02-项目管理基础.pdf
PPT/02-01 到 02-05 需求工程课件
PPT/03-01 到 03-04 体系结构与人机交互课件
PPT/04-01 详细设计.pdf
PPT/04-02 面向对象的模块化与信息隐藏.pdf
PPT/设计模式.pdf
PPT/代码设计.pdf
PPT/软件构造.pdf
软工二课程整理/课程框架与重点名词.md

2 一、总图：先记住软件工程活动链

2.1 1. 总体顺序

现实问题
-> 需求工程
-> 体系结构设计
-> 详细设计
-> 构造、测试、重构
-> 交付、维护、演化

这条线是最重要的。名词解释题大多不是孤立考定义，而是考你能否判断一个概念属于哪个阶段、解决什么问题。

2.2 2. 各阶段一句话

- 需求工程：把现实世界问题变成可验证、可追踪的规格。
- 体系结构：决定系统大的部件、连接方式和约束。
- 详细设计：把架构细化成类、方法、接口和对象协作。
- 软件构造：把设计变成可运行、可测试、可维护的代码。
- 项目管理：在范围、时间、成本和质量之间做控制。
- AI 协作：生成初稿可以交给 AI，工程判断必须由人负责。

3 二、为什么需要软件工程

3.1 记忆链

程序只是代码

-> 软件 = 程序 + 数据 + 文档 + 知识

-> 软件复杂后出现软件危机

-> 所以需要软件工程的方法、过程和质量控制

软件

软件是程序、数据、文档和知识的集合，不只是可执行代码。

软件工程

软件工程是把系统化、规范化、可度量的方法应用于软件开发、运行和维护，并研究这些方法的学科。目标是在受控成本、进度和质量下交付可维护的软件系统。

软件危机

软件规模和复杂度增长后，传统个人式开发难以控制成本、进度、质量和维护，导致延期、超支、缺陷多和难维护。

软件生命周期

软件从提出、需求、设计、构造、测试、部署、运行维护到退役的全过程。

软件维护

软件交付后修改系统或部件，以修正缺陷、改进性能或适应环境变化。

正向工程

从需求和设计出发构造代码、文档和可运行系统的过程。

逆向工程

从已有系统分析其部件、关系、需求和设计，用更高层抽象描述系统。

再工程

在理解旧系统基础上重构、迁移或重新实现，以改善结构、质量、可维护性或适配新环境。

4 三、项目管理与过程控制

4.1 记忆链

项目有目标和边界

-> 项目受范围、时间、成本约束

-> 过程模型组织开发节奏

-> 风险、质量和配置管理保证项目可控

项目

为创建唯一产品、服务或成果而进行的临时性工作，具有明确开始和结束。

铁三角

项目管理中的范围、时间、成本三类约束，三者共同影响软件质量。

风险管理

对不确定事件及其影响进行识别、分析、排序、应对和监控。

Build-and-Fix Model

构建-修补模型。开发者直接构建软件并不断修补，没有规范过程、文档和质量控制；系统变复杂后会导致结构退化和维护困难。

Waterfall Model

瀑布模型。需求、设计、实现、测试、交付按阶段顺序推进，适合需求稳定、合同边界明确的项目。

Incremental Model

增量模型。把系统按功能优先级拆成多个增量，每个增量交付一部分可运行功能。

Iterative Model

迭代模型。通过多轮开发逐步完善系统，每轮都吸收反馈并修正上一轮不足。

Evolutionary Model

演化模型。先交付初始版本，再根据用户反馈、环境变化和新需求持续修改。

Prototype Model

原型模型。用原型澄清界面、流程或业务规则，适合用户难以准确表达需求的场景。

Spiral Model

螺旋模型。风险驱动的迭代模型，每一轮包括目标设定、风险分析、开发验证和计划下一轮。

SQA

软件质量保障。通过需求评审、设计评审、代码审查、静态分析、测试、度量和过程审计保证过程和产品满足质量要求。

SCM

软件配置管理。用于管理软件制品及其变更，活动包括配置项识别、版本管理、变更控制、配置审计、状态报告和软件发布管理。

配置项

纳入配置管理的项目制品，例如需求文档、设计文档、源代码、测试脚本、配置文件、数据库脚本和部署脚本。

基线

经过正式评审和批准的规格或产品版本，后续变更必须经过变更控制。

变更控制

对变更请求进行提交、影响分析、批准或拒绝、实施、验证和状态更新的过程。

CI

持续集成。开发者频繁把代码合并到主干，由自动构建、自动测试和质量检查及时发现集成问题。

CD

持续交付或持续部署。用自动化流水线把构建、测试、制品管理和发布过程做成可重复、可审计、可回滚的流程。

用户文档

面向最终用户的文档，说明安装、登录、导航、操作步骤、输入格式、错误恢复和常见问题。

系统文档

面向维护和运维人员的文档，说明软硬件配置、网络、安全、备份、部署、故障处理和部件替换。

5 四、需求工程：把现实问题变成规格

5.1 记忆链

问题域

- > 需求获取
- > 需求分析
- > 规格说明
- > 需求验证
- > 需求管理

需求部分最容易混淆的是层次和类型。层次回答“站在谁的角度”，类型回答“约束什么内容”。

需求

用户为解决问题或达到目标所需要的条件或能力，也包括这种条件或能力的文档化表述。

需求工程

所有需求处理活动的总和，包括需求获取、需求分析、规格说明、需求验证和需求管理。

问题域

需求产生和被解决的现实世界环境，包括领域知识、利益相关人、现有流程和业务约束。

业务需求

高层目标，回答为什么要建设系统，体现组织目标和业务价值。

用户需求

用户希望通过系统完成任务，描述系统能帮助用户做什么。

系统级需求

面向软件产品的精确规格说明，描述系统在输入、事件或约束下必须表现出的可验证行为。

功能需求

系统要做什么，描述系统必须提供的功能和业务行为。

非功能需求

系统要达到什么程度，描述性能、安全、可靠性、可维护性、易用性等质量属性和约束。

性能需求

对响应时间、吞吐量、容量、并发、负载和实时性的要求。

质量属性

系统质量方面的要求，例如可靠性、安全性、可用性、可维护性、可移植性和易用性。

对外接口需求

规定系统与外部系统、设备、网络、用户界面之间交互方式、输入输出、协议和异常处理的需求。

约束

系统构造和运行必须遵守的条件，例如技术栈、法律法规、平台环境、行业标准和业务规则。

数据需求

规定系统需要保存的数据字段、关系、完整性约束、访问频率、保留时间和数据格式。

用户故事

按“作为某角色，我希望某功能，以便某价值”的格式描述需求，并配合验收标准。

INVEST

好用户故事的准则：独立、可协商、有价值、可估算、小粒度、可测试。

用例

系统与外部参与者交互中，为参与者提供有价值服务的一组行为序列。

场景

用例中的单条路径。用例包含主成功场景和多个扩展场景。

Actor

与系统交互的外部角色，可以是人、外部系统、硬件设备或定时器。

系统边界

划定系统内外范围。Actor 在边界外，用例在边界内。

Include

包含关系。一个用例每次执行都必须包含另一个公用例的行为。

Extend

扩展关系。一个用例在特定条件下扩展另一个基础用例。

概念类图

描述应用领域中重要概念、属性、关联和多重性的模型，也称领域模型。

系统顺序图

把系统当成黑盒，描述一次用例中参与者、系统和外部系统之间消息传递的时间顺序。

状态图

描述对象在事件触发下从一个状态转换到另一个状态的模型，适合生命周期明显的业务对象。

OCL

对象约束语言，用于精确表达 UML 图难以表达的约束。

SRS

软件需求规格说明文档。从系统视角以可验证、可追踪、可修改的系统级需求列表描述软件解决方案。

需求验证

检查需求是否正确、完整、一致、可读、可修改、可验证和可实现。

需求追踪

记录需求与设计、代码、测试之间的对应关系，用于一致性审计和变更影响分析。

需求冲突

需求之间存在矛盾、资源竞争、优先级差异或时序冲突。

Spec-Driven Development

规格驱动开发。把规格说明作为首要产物，后续设计、任务拆分、代码和测试从规格推导。

6 五、体系结构与人机交互：把规格组织成系统

6.1 记忆链

需求

-> 设计

-> 分解、抽象、关注点分离

-> 部件、连接件、配置

-> 架构风格和架构决策

-> HCI 保证用户能有效使用系统

软件设计

关于软件对象的设计活动，既指软件对象实现的规格说明，也指产生规格说明的过程。

分解

把大问题拆成子系统、包、类、方法等更小单位，降低复杂度。

抽象

隐藏内部细节，只暴露必要接口，使人关注本质规格而不是实现细节。

关注点分离

每次只关注系统一个维度，通过分层和模块化隔离不同变化原因。

软件体系结构

系统的部件、连接件和配置。它规定系统计算部件以及部件之间的交互。

部件

承载系统主要功能和状态的组成单位，例如包、类集合、服务或进程。

连接件

定义部件之间交互方式的抽象表示，例如接口、事件、消息、REST 调用。

配置

部件和连接件之间的组织方式和拓扑结构。

体系结构风格

一组设计决策，规定组件类型、连接方式和拓扑约束。

主程序/子程序风格

以过程调用组织系统。优点是流程清楚、控制强；缺点是调用耦合强，接口变化影响大。

面向对象风格

用对象封装数据和行为，通过消息协作。优点是可实现修改、结构贴近领域；缺点是仍存在接口耦合、标识耦合和副作用。

分层架构

将系统分为展示层、业务逻辑层、数据访问层等，上层依赖下层或下层接口，适合企业信息系统。

MVC

Model、View、Controller 分离。Model 管理数据和业务逻辑，View 展示，Controller 处理输入和协调。

微服务

将系统拆成可独立部署、独立扩展的小服务。它提升团队自治和扩展性，但运维、通信和一致性复杂度高。

ASR

架构重要需求，会影响架构决策的需求，例如并发、安全、可用性、性能、技术约束。

ADR

架构决策记录，记录背景、决策、理由、被拒绝方案和后果。

架构气味

架构中的结构性问题，例如循环依赖、God Component、层违规调用和不清晰边界。

Conway 定律

系统架构会受到组织沟通结构的约束。

REP

重用发布等价原则。重用粒度等于发布粒度。

CCP

共同封闭原则。同一包内类应对同类变化共同封闭，变化应尽量限制在少数包内。

CRP

共同重用原则。同一包内类应被一起重用，避免强迫使用者依赖不需要的类。

ADP

无环依赖原则。包依赖关系图不能存在环。

SDP

稳定依赖原则。包依赖应指向更稳定的包。

SAP

稳定抽象原则。稳定包应更抽象，不稳定包应更具体。

POJO

Plain Old Java Object。普通 Java 对象，常用于表达实体或简单数据结构。

VO

值对象或视图对象。常用于逻辑层向展示层传递数据。

可用性

用户在指定环境中有效、高效、满意地完成任务的程度，维度包括易学性、效率、易记性、错误率和满意度。

精神模型

用户对任务和系统工作方式的内在理解。界面设计应贴近用户精神模型。

菲茨定律

点击目标越大、越近，完成指向动作越快。用于指导按钮尺寸和位置设计。

导航

帮助用户理解当前位置、可去位置和返回路径的界面机制。

反馈

系统对用户操作及时给出结果、状态或错误信息，使用户知道操作是否成功。

协作式设计

让用户和利益相关人参与界面和交互设计，通过反馈提升可用性和任务匹配度。

7 六、详细设计与模块化：把架构落到类和对象

7.1 记忆链

架构给出大结构

- > 详细设计分配职责
- > 对象通过协作完成行为
- > 模块化追求高内聚、低耦合
- > 信息隐藏和封装隔离变化

详细设计

将体系结构决策精化到可直接编码的类、方法、数据结构、算法和控制结构。

职责

类或对象要执行的任务或维护的数据，包括操作职责和数据职责。

协作

多个对象通过消息传递共同完成某个行为。

委托

一个对象保留高层职责，把具体执行交给更专门的对象。

GRASP

通用职责分配原则，用于指导面向对象设计中的职责分配。

Low Coupling

低耦合。分配职责时尽量减少模块依赖。

High Cohesion

高内聚。让类职责集中，只承担紧密相关的一组职责。

Information Expert

信息专家原则。把职责分配给拥有完成该职责所需信息的类。

Creator

创建者原则。由聚合、包含、记录、紧密使用或持有初始化数据的类负责创建对象。

Controller

控制器原则。为系统事件选择合适的处理对象，避免表现层直接调用领域对象。

集中式控制

少数控制器记录大部分系统行为，决策位置清晰，但控制器容易膨胀。

委托式控制

控制器保留主流程，把子任务委托给 Service、领域对象、策略对象和 Repository。

分散式控制

系统行为广泛分散在对象网络中，控制流难理解，通常会降低可维护性。

模块

可独立设计、实现和测试的软件单元。

耦合

模块之间依赖关系的复杂程度。设计目标是低耦合。

内容耦合

一个模块直接访问或修改另一个模块内部数据或代码，耦合程度最高。

公共耦合

多个模块依赖同一全局数据或共享状态。

控制耦合

调用方传递控制标志或类型码，让被调用方选择行为。

印记耦合

模块传递整个结构或对象，但被调用方只使用其中一部分。

数据耦合

模块之间只传递必要数据参数，耦合较低。

内聚

模块内部元素的关联程度。设计目标是高内聚。

功能内聚

模块内部所有元素共同完成一个明确功能，是高内聚形式。

信息内聚

模块围绕同一数据结构组织多个操作。

通信内聚

模块内操作使用同一组输入或输出数据。

过程内聚

模块内元素按流程顺序组织，但不一定服务于单一职责。

逻辑内聚

模块根据类型参数或标志选择不同逻辑，通常需要拆分。

偶然内聚

模块内元素无明确关联，是低内聚形式。

信息隐藏

每个模块隐藏一个重要设计决策，把变化限制在模块内部。

封装

通过接口隔离实现细节，包含封装数据和行为、内部结构、内部对象引用、类型信息和潜在变更。

全局变量有害

共享全局变量造成公共耦合，使状态变化难追踪，降低可测试性和可维护性。

显式化原则

把依赖、输入、输出和约束显式表达，减少隐式假设。

DRY

不要重复。重复代码或重复规则会导致修改遗漏和不一致。

面向接口编程

依赖接口而非具体实现，以降低耦合并支持替换、测试和扩展。

Law of Demeter

迪米特法则。只与直接朋友通信，避免通过对象链访问陌生对象。

ISP

接口隔离原则。调用方不应被迫依赖它不用的方法。

LSP

里氏替换原则。子类型必须能替换父类型而不改变程序正确性，并维护父类契约和不变式。

组合优于继承

优先通过对象组合和委托复用行为，减少对子类和父类实现细节的依赖。

SRP

单一职责原则。一个类或模块只保留一个变化原因。

OCP

开闭原则。软件实体应对扩展开放，对修改关闭。

DIP

依赖倒置原则。高层模块和低层模块都依赖抽象，抽象不依赖细节。

SOFA

方法级设计原则。Short、One thing、Few arguments、Abstraction level consistency。

8 七、设计模式：用稳定抽象隔离变化

8.1 记忆链

设计模式不是背类图

-> 先找变化点

-> 再用稳定接口隔离变化

-> 最后说明代价

设计模式

对反复出现的设计结构和协作方式的经验抽象，用于提升灵活性、复用性和可维护性。

Strategy

策略模式。定义一族算法并分别封装，使它们可以相互替换，让算法变化独立于使用算法的客户。

Abstract Factory

抽象工厂模式。提供创建一组相关对象的接口，将具体创建逻辑封装在具体工厂中。

Factory Method

工厂方法模式。父类定义创建接口，由子类决定实例化哪个具体类。

Singleton

单件模式。确保一个类只有一个实例，并提供全局访问点。工程实践中也常由依赖注入容器管理单例生命周期。

Iterator

迭代器模式。顺序访问聚合对象的元素，而不暴露聚合对象内部表示。

Observer

观察者模式。一个对象状态变化后通知多个观察者对象，适合界面显示和订阅通知。

DAO

数据访问对象模式。用接口封装持久化访问，使业务层不依赖具体数据库或存储技术。

9 八、软件构造、代码质量与测试

9.1 记忆链

详细设计
-> 编码

- > 单元测试和集成测试
- > 调试和代码评审
- > 重构和回归测试

软件构造

通过编码、验证、单元测试、集成测试、调试、代码评审和构建活动生产可工作的软件。

可读性

代码应清楚表达意图，便于开发、调试、维护和复用。

可维护性

代码容易修改，复杂度受控，依赖关系清晰，测试和文档能支持变更。

契约式设计

用前置条件、后置条件和不变式描述方法或类的使用契约。

防御式编程

面对外部输入、环境和依赖对象时主动检查错误，保护内部逻辑。

断言

在代码中检查必须成立的条件，用于发现程序内部假设被破坏。

异常处理

对运行时异常情况进行捕获、传播、恢复或转换，使错误处理路径明确。

调试

定位和修复错误的过程，包括重现问题、诊断缺陷、修复缺陷和验证修复。

代码评审

对代码进行系统检查，发现逻辑、安全、规范、可读性和可维护性问题。

重构

不改变代码外部行为的前提下改善内部结构，目标是提升可读性、可维护性、可测试性和扩展性。

坏味道

表明代码需要重构审查的迹象，例如过长方法、大类、过多参数、重复代码、复杂条件和魔法数字。

TDD

测试驱动开发。先写失败测试，再写最小实现让测试通过，最后重构。

Red-Green-Refactor

TDD 三阶段：先让测试失败，再写代码让测试通过，再重构改善结构。

结对编程

两个程序员共同协作进行构造活动，并轮换驾驶员和观察员角色。

表驱动

用表、Map 或规则对象表达条件到动作的映射，减少复杂分支。

单元测试

测试单个类、方法或模块是否满足预期，常使用 Mock 隔离外部依赖。

集成测试

测试多个模块之间的接口和协作是否正确。

回归测试

修改后重新测试已有功能，防止旧功能被破坏。

Stub

被测模块依赖的下层临时代码，用固定返回值替代真实依赖。

Driver

调用被测模块的上层临时代码，用来驱动测试执行。

Mock Object

可验证交互的替身对象，既能模拟依赖返回值，也能记录依赖是否按预期被调用。

黑盒测试

把测试对象看成黑盒，根据规格说明设计测试用例，不关注内部程序结构。

等价类划分

把输入划分为有效类和无效类，从每类选取代表值设计测试。

边界值分析

围绕输入或输出边界、边界内、边界外取值，发现闭开区间和极值错误。

决策表测试

针对多个条件组合决定多个动作的场景，用条件、动作和规则组合设计测试。

状态转换测试

针对对象行为依赖当前状态的场景，覆盖有效转换和重要无效转换。

白盒测试

把测试对象看成透明结构，根据程序内部控制流和数据流设计测试用例。

语句覆盖

每条语句至少执行一次。

分支覆盖

每个判定的真分支和假分支至少执行一次。

条件覆盖

每个基本条件的真值和假值至少出现一次。

路径覆盖

每条独立执行路径至少执行一次。

圈复杂度

度量程序控制流复杂度。常用公式是 $V(G) = \text{判定节点数} + 1$ 。

10 九、AI 协作、工具链与 Web 实现

10.1 记忆链

AI 不是单独一章

-> 它贯穿需求、设计、编码、测试

-> 关键不是生成，而是审查、验证和承担责任

K-A-S-D

课程评价框架。K 是知识，A 是 AI 协作与判断，S 是工程技能，D 是过程与态度。

Prompt Engineering

提示词工程。通过角色、任务、约束、格式控制 AI 输出，使输出更符合需求、设计和工程约束。

AI 输出审查

对 AI 生成内容进行人工检查，重点识别错误、遗漏、歧义、冲突、幻觉和不符合工程约束的部分。

Human-in-the-Loop

人在环。AI 负责生成、分析或辅助修改，人负责审查、决策、验证和承担最终责任。

VibeCoding

用自然语言和 AI 工具驱动原型、前端、后端、调试和移动端开发。它提高开发速度，但必须保留需求验证、代码审查、测试和安全检查。

REST

以资源为中心设计接口，用 URL 表示资源，用 HTTP 方法表达操作，强调无状态和统一接口。

JSON

前后端和系统之间交换结构化数据的常用文本格式，适合 REST API 的请求体和响应体。

Spring Boot

Java Web 后端开发框架，用自动配置、依赖管理和内嵌服务器简化应用开发。考试中常与 Controller、Service、Repository 三层职责一起出现。

JWT

JSON Web Token。用于在无状态 HTTP 请求中携带身份声明，常由客户端保存并随请求发送给服务器验证。

11 十、卷面速记：相似概念判定句

11.1 需求层次

业务需求回答为什么建设系统。

用户需求回答用户希望系统帮助完成什么任务。

系统级需求回答系统在具体输入、事件和约束下必须怎样响应。

11.2 需求类型

系统做什么 是功能需求。

系统做到什么程度 是非功能需求。

字段、格式、保留时间 是数据需求。

和外部系统通信 是对外接口需求。

技术栈、法规、平台 是约束。

11.3 用例和图

Actor 在系统边界外，用例在系统边界内。

场景是一条路径，用例是一组场景。

概念类图只画问题域概念，不画 **Controller**、**Service**、**Repository**。

系统顺序图把系统当黑盒。

详细顺序图展开 **Controller**、**Service**、**Repository**、领域对象和外部接口。

11.4 架构和详细设计

体系结构关心部件、连接件、配置。

详细设计关心类、方法、接口、职责和对象协作。

架构风格是整体组织方式，设计模式是局部对象协作方案。

11.5 模块化

低耦合看模块之间依赖少不少。

高内聚看模块内部职责集中不集中。

信息隐藏隐藏变化原因。

封装用接口隔离实现细节。

11.6 测试

黑盒测试按规格设计用例。

白盒测试按程序结构设计用例。

Stub 代替下层依赖。

Driver 调用被测模块。

Mock 既模拟依赖又验证交互。

回归测试防止修改破坏旧功能。